PRD-Based Prompting Guide for Vibe Coding Tools

A Comprehensive Reference for Lovable, v0, Bolt.new, Replit, and Firebase Studio

---

1. Overview: Why PRD-to-Prompt Matters

Vibe coding tools turn natural language into working applications. But the quality of output is directly proportional to the quality of input. A well-structured PRD (Product Requirements Document) is the single most important asset you can bring to any vibe coding tool.

The core principle across all tools is the same:

> The more specific and structured your input, the fewer iterations you need — saving time, tokens, and frustration.

Each tool has its own context management system (Knowledge files, System prompts, Project settings) that acts like a persistent memory layer. When combined with PRD-driven prompting, these systems ensure consistency across every generation.

---

2. Universal Prompting Principles (All Tools)

Before diving into tool-specific details, these principles apply everywhere:

2.1 Start with Architecture, Not Features

Never dump your entire PRD into a single prompt. Instead:

1. **Establish the skeleton first** — project structure, tech stack, layout
2. **Add features incrementally** — one feature per prompt, test, then next
3. **Polish last** — animations, transitions, copy refinement

Bad: "Build a complete e-commerce platform with auth, catalog, cart, payments, orders, admin dashboard, and analytics"

Good: "I'm building an e-commerce platform for handmade crafts. Let's start with the product listing page only. Use Next.js 14, TypeScript, Tailwind CSS, and shadcn/ui. Show a responsive grid of product cards with image, title, price, and 'Add to Cart' button."

2.2 The Three-Input Framework (from v0's Official Guide)

Every effective prompt contains three core inputs:

Input	Description	Example
Product Surface	What components, data, and actions exist	"Dashboard with KPI cards, line chart, data table"
User Context	Who uses it, how, and where	"Field sales reps on mobile, checking between meetings"
Constraints	What NOT to do, style limits, tech requirements	"Mobile-first, max 2 columns, high contrast, no external APIs"

2.3 Be Explicit About What NOT to Change

When iterating, always specify:

- Which files/components should remain untouched
- Which styles should carry forward
- Which behaviors should persist

Example: "Add a pricing section below the hero. Use the same color palette and typography. Do not modify the hero section, navigation, or any shared layout components."

2.4 Name Everything Deliberately

Labels like "Inbox," "Submit," or "Dashboard" become internal references, routes, and component names. Changing them later creates cascading issues across file paths, nav logic, and UI copy.

2.5 Use Plan/Discuss Mode Before Build Mode

Most tools offer a planning mode that doesn't modify code:

- **Lovable:** Plan Mode (use 60-70% of your time here)
- **Bolt.new:** Discuss Mode
- **Replit:** Plan Mode
- **v0:** Chat without generating
- **Firebase Studio:** Chat in Code view

Always plan first, implement when satisfied.

---

3. Tool-by-Tool Guide

3.1 Lovable

Tech Stack: React + Vite + TypeScript + Tailwind CSS + shadcn/ui + Supabase

Knowledge File (Persistent Context)

Lovable's Knowledge Base is your project's brain. It gets sent with every prompt automatically.

How to access: Settings icon → Manage Knowledge

What to include in Knowledge:

```
Project: [App Name] - [one-line description]
Tech Stack: React, Vite, TypeScript, Tailwind CSS, shadcn/ui
Backend: Supabase (PostgreSQL)
Auth: Email/password + Google OAuth

Design System:
- Primary color: #2563EB (blue-600)
- Font: Inter for body, Montserrat for headings
- Border radius: 8px for cards, 6px for buttons
- All UI must be responsive and mobile-friendly

User Roles:
- Admin: Full access to all features
- User: Can only view/edit their own data
- Viewer: Read-only access

Coding Standards:
- Use TypeScript strict mode
- Components in src/components/
- Pages in src/pages/
- Use React Query for data fetching
- All forms must have validation
```

PRD-to-Prompt Workflow

Step 1: Generate Knowledge from PRD

```
Generate knowledge for my project based on the following PRD:
[paste PRD summary - tech stack, roles, key features, design rules]
```

Step 2: Initial Scaffold

```
Create the basic layout for [App Name]:
- Navigation sidebar with: Dashboard, Projects, Settings
- Top bar with user avatar and notifications
- Main content area with breadcrumbs
- Use the design system from Knowledge
- Mobile: Collapse sidebar into hamburger menu
```

Step 3: Feature-by-Feature

```
On page /dashboard, implement the analytics overview:
- 4 KPI cards: Total Users, Active Projects, Revenue, Growth Rate
- Line chart showing monthly trends (use recharts)
- Recent activity feed (last 10 items)
- Do not touch the sidebar, navigation, or layout components
```

Key Lovable Tips

1. **Pin stable versions** — After every working feature, pin it as a checkpoint
2. **Use Select tool** for quick visual edits instead of prompts (it's free and fast)
3. **Screenshots as context** — Attach screenshots for bug reports or UX reference
4. **Guardrails in every prompt** — "Do not edit /shared/Layout.tsx"
5. **Version comparison** — "Compare version at T-1 to T-0. What changed? What might be breaking?"
6. **Role-specific prompts** — "As an Investor, I want to view the company dashboard, but I shouldn't be able to edit it"
7. **Recovery prompt** — "Please investigate this without breaking other features. If needed, revert to the last working version and fix from there."

⚠️ Lovable Gotchas

- Supabase does NOT revert cleanly — if you revert a version, your database schema may break
- After major edits, always recheck multiple roles and their behavior

- The AI's memory can be limited — repeat important instructions across prompts
- Use `generate.new/[prompt]` in browser to instantly start a new project

3.2 v0 (by Vercel)

Tech Stack: React + Next.js App Router + Tailwind CSS + shadcn/ui

Project Context / Knowledge

v0 uses **Context** (set during project creation, editable in Project Settings → Knowledge).

What to include:

```
This is [App Name], a [description] built with Next.js 14 App Router.

Design Language:
- Professional but approachable
- Card-based layouts with subtle shadows
- Color palette: [specific hex codes]
- Typography: [specific fonts]

Technical Constraints:
- All components must be accessible (ARIA labels, keyboard navigation)
- Mobile-first responsive design
- Use shadcn/ui components exclusively
- No external icon libraries (use Lucide React only)
```

The v0 Prompt Template

From Vercel's official guide, the optimal prompt structure:

```
Build [PRODUCT SURFACE: specific components, data, actions].

[USER CONTEXT: who uses it, their role, technical comfort, environment].

[CONSTRAINTS: visual style, responsive rules, color coding, what NOT to include].
```

Example:

```
Build a support ticket dashboard showing:
- Open tickets count with priority breakdown
- Filterable ticket list with status, assignee, and SLA timer
- Quick-action buttons: Assign, Escalate, Close

Users: Customer support agents working on desktop monitors.
They handle 50+ tickets/day and need information density.

Constraints:
- Desktop-first, professional dark theme
- Color code by priority: red for urgent, yellow for medium, green for low
- Show agent status badges (busy/available)
- High contrast for readability
- Maximum 3 columns
```

Iteration Approach

v0 offers two ways to iterate after generation:

1. **Prompt for changes** — Best for functional changes, adding features, restructuring
2. **Design Mode** — Click elements visually, adjust colors/spacing/typography directly

Rule of thumb: Use prompts for logic and structure. Use Design Mode for visual tweaks.

Key v0 Tips

1. **Provide reference URLs** — v0 can screenshot and clone websites you share

2. **Specify tech explicitly** — "using react-three-fiber" for 3D, specific libraries for charts
3. **10 seconds more specificity = 110 fewer lines of code** (from Vercel's testing)
4. **Use** **Add to codebase** button to generate an npx command for local Next.js project
5. **Good prompts are 30-40% faster** with less unnecessary code and fewer credits spent
6. **v0 is tightly coupled to shadcn** — it will default to shadcn even if you request other component libraries
7. **Start from community** — Fork popular projects from v0's homepage as starting points

⚠️ **v0 Gotchas**

- v0 is primarily frontend — complex backend logic needs separate implementation
- Single-file output — all code goes into one file per component
- shadcn/ui is essentially hardcoded — requesting other UI libraries may cause issues
- Generated code needs review for accessibility compliance

3.3 Bolt.new (by StackBlitz)

Tech Stack: JavaScript/TypeScript ecosystem, browser-based WebContainer

Project Prompt & System Prompt

Bolt has TWO prompt layers:

Type	Scope	Access
Project Prompt	Current project only	Gear icon → All Project Settings → Project Knowledge
System Prompt	Every project	Gear icon → All Project Settings → System Knowledge

Additionally, there's `.bolt/prompt` file in the codebase itself (edit in StackBlitz, reopen in Bolt).

Example System Prompt (applies to all projects):

```
For all designs, make them beautiful, not cookie cutter.
Make webpages that are fully featured and worthy for production.
By default, use JSX with Tailwind CSS, shadcn/ui, React hooks,
and Lucide React for icons.
Do not install other packages for UI themes unless I request them.
Use stock photos from unsplash where appropriate.
Only change relevant code — do not modify unrelated files.
```

Example Project Prompt:

```
Project: TaskFlow — A team task management app
Stack: Next.js 14, TypeScript, Tailwind CSS, Supabase
Auth: Email/password + Google OAuth via Supabase

Key Rules:
- All data operations go through Supabase client
- Use React Query for server state
- Components follow atomic design (atoms/molecules/organisms)
- Mobile-first responsive design
- Dark mode support required
```

PRD-to-Prompt Workflow

Step 1: Don't code first — Generate a PRD

This is a critical Bolt best practice. Before building, ask Bolt to create a PRD:

```
I want to build [app description]. Before writing any code,
create a detailed PRD outlining:
- Feature set and user stories
- Data model / schema
- Page structure and routing
- Component hierarchy
- API endpoints needed

Do NOT write any code yet. Just the PRD.
```

Review and refine the PRD. THEN proceed to implementation.

Step 2: Scaffold with design vibe

```
Let's start building TaskFlow. Begin with ONLY the hero section
of the landing page. Establish the color palette, typography,
button styles, and spacing system here.

Do not add pricing, features list, or navigation yet.
```

Step 3: Extend consistently

```
Using the same style and design language from the hero section,
add a features grid section below it. Show 6 features in a
responsive grid (3 columns desktop, 1 column mobile).
```

Enhance Prompt Feature

Bolt has a built-in **Enhance Prompt** button (sparkle icon) that automatically expands your prompt into a PRD-like specification. Always review the enhanced version before submitting — it may drop specific details.

Key Bolt Tips

1. **Break requests into small parts** — One change → verify → next change
2. **Batch SIMPLE instructions** — Color change + mobile responsive + restart can go in one prompt

3. **Reference specific code** — "In the `handleSubmit` function in `LoginForm.tsx`..."
4. **Coding standards via `.bolt/prompt`** — Define camelCase, component structure, etc.
5. **Import from GitHub** — Fork repo to StackBlitz first, then import to Bolt
6. **Summarize before reset** — Ask Bolt to summarize conversation, then duplicate project to reset chat history
7. **Lock/Target** — Use these features for precise code updates to specific sections
8. **Open preview in new tab** — Required for Stripe, OAuth, YouTube embeds (iframe blocking)

⚠️ Bolt Gotchas

- **No Python/C#/C++** — Web languages only (JS/TS/WebAssembly)
- **No pip support** — Only npm/yarn packages
- **Chat history is finite** — Long conversations degrade quality; summarize and reset
- **Tokens = money** — Complex prompts cost more; be concise and precise
- **Env variables** — Bolt may expose secrets; use `.env` files and secure before deployment
- **Enhanced Prompt may alter details** — Always review before submitting

---

3.4 Replit

Tech Stack: 50+ languages, full Linux container, Python/JS/Go/C++ and more

Custom Agent Instructions (replit.md)

Replit uses a `replit.md` file for persistent project context.

What to include:

```markdown
```

```
# Project: [App Name]

## Overview
[One paragraph description]

## Tech Stack
- Frontend: React with TypeScript
- Backend: Python FastAPI
- Database: Replit Database / PostgreSQL
- Auth: Replit Auth or custom JWT

## Coding Style
- Python: Follow PEP 8, use type hints
- TypeScript: Strict mode, functional components
- Use async/await everywhere
- Error handling: Always use try/catch with specific error types

## Project Structure
src/
├── frontend/    # React app
├── backend/     # FastAPI server
├── shared/      # Shared types and utilities
└── tests/       # Test files

## Key Rules
- Always write tests for new features
- Use environment variables for secrets
- Follow REST API naming conventions
- All API responses must include proper error codes
```

PRD-to-Prompt Workflow

Step 1: Plan before building

```
I'm building [app description]. Before writing any code:
1. Outline the features and data structures
2. Propose the tech stack
3. Map out the user flows
4. Suggest the project structure

Let me review before we start implementing.
```

Step 2: Incremental implementation

```
Step 1: Create the basic HTML structure for a task manager
with an input field and a task list.
```

→ Create checkpoint → Test → Next step

```
Step 2: Add JavaScript to allow users to add tasks to the list.
```

→ Create checkpoint → Test → Next step

```
Step 3: Use Replit Database to store tasks so they persist.
```

Step 3: Debug with context

Bad: "It's broken"

Good: "The login form submission is failing with a 400 error when using valid credentials. The error occurs in `auth.py` line 42. Here's the error log: [paste error]"

Key Replit Tips

1. **Use Plan Mode** — Ask Agent for advice before implementing
2. **Checkpoints are your safety net** — Create one after every working feature
3. **Rollback instantly** — If something breaks, rollback to last checkpoint
4. **Attach files** — Upload API docs, screenshots, or code examples as context

5. **"Improve Prompt" button** — Rewrites basic ideas into detailed, AI-friendly instructions
6. **One feature at a time** — Cheaper and less error-prone than requesting all at once
7. **Queue & Plan Mode** — For managing longer, multi-step tasks systematically
8. **Fast Mode** — Enable for faster execution with cached dependencies

⚠️ Replit Gotchas

- **Effort-based billing** — Complex prompts cost more; break into small steps
- **Agent loops** — If Agent gets stuck, rollback and rephrase
- **Full-stack complexity** — More capable but also more things that can go wrong
- **Production readiness** — Good for MVPs but may need additional hardening for production

---

3.5 Firebase Studio (by Google)

Tech Stack: Next.js (App Prototyping Agent), any framework (Code view), Gemini AI

AI Rules Configuration

Firebase Studio uses AI Rules files for persistent context:

Supported files (checked in order):

- `.idx/airules.md`
- `.cursorrules`
- `.github/copilot-instructions.md`
- Other AI rules files

Example `.idx/airules.md`:

```markdown
```

```
# Project Rules

## App Overview
[App Name] is a [description] built with Next.js 14.

## Design Principles
- Mobile-first responsive design
- Accessibility: WCAG 2.1 AA compliance
- Performance: Core Web Vitals targets

## Tech Constraints
- Use Tailwind CSS for styling
- shadcn/ui for components
- Gemini API for AI features (key auto-provided)
- Firebase Auth for authentication
- Firestore for database

## Code Style
- TypeScript strict mode
- Functional components only
- Custom hooks for shared logic
- Server components by default, client only when needed
```

Two Modes of Development

Mode	Use Case	Best For
App Prototyping Agent	No-code, prompt-driven	Quick prototypes, MVPs, non-coders
Code View	Full IDE (Code OSS)	Custom logic, complex integrations, experienced devs

You can switch between modes seamlessly — start with Prototyping Agent, refine in Code view.

PRD-to-Prompt Workflow

Step 1: Structured initial prompt

```
Create a [type] app called [Name]:

Purpose: [what it does and for whom]
Target audience: [who uses it]

Core Features:
1. [Feature with specific behavior]
2. [Feature with specific behavior]
3. [Feature with specific behavior]

Design: [visual style, colors, mood]
Backend: [Firebase Auth + Firestore / other services]

Start with the main screen only.
```

Step 2: Iterate with specifics

```
Add user authentication:
- Sign up with email/password
- Google OAuth sign-in
- Protected routes for /dashboard and /settings
- Redirect unauthenticated users to /login

Also tell me what Firebase services need to be set up
and guide me through the configuration.
```

Step 3: Use multimodal inputs

- Upload images/screenshots as design reference
- Use the built-in drawing tool to sketch layouts
- Paste Figma designs via Builder.io plugin

Key Firebase Studio Tips

1. **Multimodal prompts** — Use images, drawings, and text together for best results
2. **Ask for setup instructions** — Gemini can write code but can't provision services; ask it to explain setup steps
3. **Enhanced prompts** — Use the "enhance prompt" feature to refine your initial idea
4. **Fork workspaces** — Create copies before risky changes (preserves chat history!)
5. **Firebase integration via prompt** — Just ask: "Add Firebase Authentication to my app"
6. **Gemini API auto-provided** — AI features work out-of-the-box without your own key
7. **AI-optimized templates** — Start with official templates for React, Next.js, Flutter, Angular
8. **Export to GitHub** — Sync your code for manual editing and deployment

⚠️ **Firebase Studio Gotchas**

- **Next.js only** for App Prototyping Agent (other frameworks coming)
- **Token limits** — Large projects can exceed model context limits
- **Firebase config values** — Include specific values in prompts (e.g., "Update measurementId with G-1234567890")
- **Preview URL for Auth** — Must add preview URL as authorized domain for authentication to work
- **Free tier limits** — 3 workspaces (10 with Google Developer Program, 30 with Premium)

---

4. PRD Section → Prompt Mapping Matrix

This matrix shows how to transform each PRD section into effective prompts across tools:

PRD Section	Universal Approach	Tool-Specific Notes
Product Vision	Include in Knowledge/Context file	Lovable: Knowledge Base · Bolt: System Prompt · v0: Project Context · Replit: replit.md · Firebase: airules.md
User Personas	State who uses it, their technical level, environment	v0: Critical for UX decisions ("field sales reps on mobile")
Tech Stack	Specify in first prompt AND Knowledge file	All tools: Prevents wrong framework choices
Data Model	Describe schema before building data-heavy features	Lovable/Bolt: Crucial for Supabase setup · Replit: Database schema prompt
Page Structure	Scaffold pages/routes before features	All tools: "Create routing for: /dashboard, /projects, /settings"
UI/UX Design	Provide colors, fonts, spacing, component library	All tools: Include hex codes, not color names
Auth & Roles	Define roles and permissions explicitly	Lovable: "As role X, I should/shouldn't see Y"
API Endpoints	List endpoints with request/response shapes	Replit/Bolt: Most capable for custom backends
Business Logic	One rule per prompt, test immediately	All tools: Complex logic = more iterations needed
Testing	Ask for test cases after each feature	Replit: Best built-in testing support
Deployment	Specify target platform early	Bolt: Netlify · Lovable: Built-in · v0: Vercel · Firebase: App Hosting

5. Prompt Templates by Use Case

5.1 Initial Project Setup

```
I'm building [APP NAME], a [TYPE] for [TARGET USERS].

Tech Stack: [FRAMEWORK], [CSS], [UI LIBRARY]
Backend: [DATABASE/AUTH PROVIDER]

Core screens:
1. [Screen name] - [brief description]
2. [Screen name] - [brief description]
3. [Screen name] - [brief description]

Design vibe: [ADJECTIVE] (e.g., minimal, vibrant, corporate, playful)
Color palette: [PRIMARY HEX], [SECONDARY HEX], [ACCENT HEX]
Font: [HEADING FONT] for headings, [BODY FONT] for body

Let's start with ONLY [FIRST SCREEN]. Do not build other screens yet.
```

5.2 Adding a Feature

```
On page [/ROUTE], implement [FEATURE NAME]:

Expected behavior:
- [Behavior 1]
- [Behavior 2]
- [Behavior 3]

This feature is for [ROLE] users only.
[ROLE2] should not see this component.
```

Do not touch: [LIST OF FILES/COMPONENTS TO PRESERVE]
Use the existing design language and color palette.

5.3 Bug Fix

Bug: [WHAT'S HAPPENING]
Expected: [WHAT SHOULD HAPPEN]
Steps to reproduce:
1. [Step 1]
2. [Step 2]
3. [Step 3]

Error message: [EXACT ERROR TEXT]
Affected component: [FILE/COMPONENT NAME]

Please investigate without breaking other features.
If needed, revert to the last working version and fix from there.

5.4 Design Refinement

The current [COMPONENT] needs visual improvement:

Current issues:
- [Issue 1: e.g., "spacing is too tight between cards"]
- [Issue 2: e.g., "button colors don't match the palette"]

Desired result:
- [Specific change: e.g., "16px gap between cards"]
- [Specific change: e.g., "buttons should use #2563EB"]

Reference: [URL or description of desired look]
Do not change functionality or data flow.

5.5 Database/Backend Setup

```
Set up the database schema for [FEATURE]:

Tables needed:
- [table_name]: [field1 (type)], [field2 (type)], ...
- [table_name]: [field1 (type)], [field2 (type)], ...

Relationships:
- [table_a].user_id → [table_b].id (foreign key)

Security rules:
- Users can only read/write their own data
- Admin can read all data
- Public can read [specific table] only

Also set up the API layer to interact with these tables.
```

---

6. Context Management Strategy

The Knowledge File Hierarchy

```
Level 1: Global/System (applies to ALL projects)
├── Coding standards
├── Preferred libraries
├── General design principles
└── Your personal style preferences

Level 2: Project-Specific (applies to current project)
├── App description and goals
├── Tech stack details
├── User roles and permissions
```

```
├── Design system (colors, fonts, spacing)
├── Data model overview
└── Key architectural decisions

Level 3: Per-Prompt Context (provided in each message)
├── Current task details
├── Specific constraints
├── Reference to relevant components
└── What NOT to change
```

When to Update Knowledge Files

- **After major architectural decisions** — New patterns, libraries, or conventions
- **When roles/permissions change** — New user types or access rules
- **After design system updates** — New colors, components, or spacing rules
- **When encountering repeated issues** — Add "Do NOT..." rules to prevent recurrence

---

7. Common Anti-Patterns to Avoid

Anti-Pattern	Why It Fails	Better Approach
One mega prompt	Overwhelms the model, inconsistent results	Break into 5-10 focused prompts
Vague descriptions	Model guesses wrong, wastes iterations	Be specific: hex codes, px values, exact behaviors
Changing names mid-project	Breaks routes, file paths, component references	Name deliberately from the start

Anti-Pattern	Why It Fails	Better Approach
Skipping Knowledge files	Repeating context in every prompt	Set up Knowledge once, reference forever
Building without testing	Errors compound and become unfixable	Test after EVERY feature addition
Not using Plan/Discuss mode	Commits to code before thinking through approach	Plan 60-70% of time, build 30-40%
Ignoring version control	No rollback when things break	Pin/checkpoint every stable version
Expecting common sense	AI is literal-minded, not intuitive	State everything explicitly
Content polish too early	Locks in details that may need to change	Structure first, content last
Fixing AI loops with more prompts	Creates deeper loops	Rollback to stable version, rephrase

8. Tool Selection Guide

If you need...	Best Tool	Why
Fastest MVP prototype	**Lovable**	Instant full-stack with Supabase, great UX out of box
Best React/Next.js components	**v0**	Deep shadcn/ui integration, highest UI quality
Full-stack web app	**Bolt.new**	Complete WebContainer, good backend support

If you need...	Best Tool	Why
Non-web or multi-language project	**Replit**	50+ languages, full Linux container
Firebase/Google ecosystem	**Firebase Studio**	Native Firebase integration, Gemini built-in
Maximum code control	**Replit** or **Firebase Studio (Code view)**	Full IDE access with AI assistance
Team collaboration	**Replit** or **Lovable**	Real-time collaboration features
Learning/education	**v0** or **Firebase Studio**	Excellent explanations and chain-of-thought

---

9. Integration with Clearly PRD Service

For Jaehee's Clearly platform, the PRD-to-Vibe-Coding-Prompt generator could offer:

Automated Prompt Generation from PRD

```
Input: Clearly PRD document
↓
Processing: Extract sections, map to tool-specific templates
↓
Output: Ready-to-paste prompts for each tool
├── Lovable: Knowledge file + initial scaffold prompt + feature prompts
├── v0: Context file + three-input structured prompts
├── Bolt.new: System prompt + project prompt + .bolt/prompt file
├── Replit: replit.md + step-by-step implementation prompts
└── Firebase Studio: airules.md + prototyping agent prompts
```

Monetization Tie-in

- **Free:** Universal initial prompt (works anywhere)
- **Pro:** Tool-specific optimized prompts + Knowledge/Context files
- **Team:** Shared prompt libraries + version-controlled Knowledge files
- **Enterprise:** Custom prompt templates per framework/industry

---

Last Updated: February 2026 Author: Built for Vibe Coding Boot Camp curriculum